

PROJET INTÉGRATEUR

TP CI/CD ET DÉPLOIEMENT

Application : Billetterie intelligente avec QR Code et abonnements

TP 1 - Intégration continue avec GitHub Actions

1. Contexte

L'équipe dispose d'une première version du projet Billetterie intelligente. Avant de poursuivre le déploiement, elle doit mettre en place une chaîne d'intégration continue permettant de vérifier automatiquement le backend et le frontend à chaque modification du code.

2. Objectifs pédagogiques

- Organiser le dépôt GitHub du projet.
- Préparer les scripts npm nécessaires aux tests et au build.
- Créer un workflow GitHub Actions.
- Automatiser les tests du backend.
- Automatiser le build du frontend.
- Utiliser GitHub Secrets pour les variables sensibles.
- Diagnostiquer un pipeline échoué à partir des logs.
- Obtenir un pipeline CI fonctionnel et stable.

3. Situation de départ

```
billetterie-intelligente/  
├── backend/  
│   ├── src/  
│   ├── tests/  
│   ├── package.json  
│   └── .env  
├── frontend/  
│   ├── src/  
│   ├── package.json  
│   └── .env  
├── README.md  
└── .gitignore
```

4. Étape 1 - Préparer le dépôt GitHub

Chaque groupe doit vérifier que le projet est prêt à être versionné et partagé.

Élément	Vérification attendue
.gitignore	Présent et complet
.env	Absent du dépôt distant
.env.example	Présent et documenté

README.md	Contient les instructions de base
package.json backend	Scripts start, dev et test disponibles
package.json frontend	Script build disponible

Exemple de fichier .gitignore :

```
node_modules
.env
dist
build
coverage
npm-debug.log
.DS_Store
```

Exemple de fichier .env.example backend :

```
PORT=
MONGO_URI=
JWT_SECRET=
EMAIL_USER=
EMAIL_PASS=
NODE_ENV=
```

Commandes Git attendues :

```
git status
git add .
git commit -m "chore: préparer le projet pour la CI"
git push
```

5. Étape 2 - Vérifier les tests backend

Le backend doit comporter des tests automatisés sur les fonctions critiques de la billetterie.

Fonctionnalité	Test attendu
Connexion utilisateur	Connexion réussie avec identifiants valides
Création utilisateur	Refus si email déjà utilisé
Création abonnement	Abonnement enregistré correctement
Génération QR Code	QR Code généré pour un abonnement actif
Scan QR Code	QR Code valide accepté
Double scan	Second scan refusé
Solde insuffisant	Scan refusé si solde à zéro
Utilisateur bloqué	Accès ou scan refusé

Script recommandé dans le package.json du backend :

```
{
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js",
  }
}
```

```

    "test": "jest --runInBand",
    "test:coverage": "jest --coverage"
  }
}

```

Commande à exécuter :

```

cd backend
npm test

```

6. Étape 3 - Créer le workflow CI backend

Créer le fichier suivant :

`.github/workflows/ci.yml`

Contenu proposé :

```

name: CI Backend Billetterie

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
  test-backend:
    runs-on: ubuntu-latest

    defaults:
      run:
        working-directory: backend

    steps:
      - name: Récupérer le code
        uses: actions/checkout@v4

      - name: Installer Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 22

      - name: Installer les dépendances
        run: npm ci

      - name: Lancer les tests
        run: npm test
        env:
          NODE_ENV: test
          JWT_SECRET: test_secret
          MONGO_URI: ${ secrets.MONGO_URI_TEST }

```

7. Étape 4 - Ajouter les secrets GitHub

Dans GitHub, aller dans Settings > Secrets and variables > Actions > New repository secret.

Secret	Utilité
MONGO_URI_TEST	Connexion à la base de test
JWT_SECRET	Signature des tokens de test
VITE_API_URL	URL du backend utilisée par le frontend
EMAIL_USER	Compte email utilisé pour les tests si nécessaire
EMAIL_PASS	Mot de passe ou clé applicative email

8. Étape 5 - Ajouter le build frontend

Compléter le fichier ci.yml avec un second job pour vérifier que le frontend compile correctement.

```
build-frontend:
  runs-on: ubuntu-latest

  defaults:
    run:
      working-directory: frontend

  steps:
    - name: Récupérer le code
      uses: actions/checkout@v4

    - name: Installer Node.js
      uses: actions/setup-node@v4
      with:
        node-version: 22

    - name: Installer les dépendances
      run: npm ci

    - name: Construire le frontend
      run: npm run build
      env:
        VITE_API_URL: ${ secrets.VITE_API_URL }
```

9. Étape 6 - Diagnostiquer un pipeline échoué

Chaque groupe doit volontairement provoquer un échec puis le corriger.

- Modifier temporairement une assertion dans un test.
- Faire un push.
- Observer l'échec dans l'onglet Actions de GitHub.
- Lire les logs du job échoué.
- Identifier la cause de l'erreur.
- Corriger le test ou le code.
- Faire un nouveau push.
- Obtenir un pipeline vert.

10. Livrables du TP 1

- Dépôt GitHub organisé.
- Fichier .github/workflows/ci.yml.
- Secrets GitHub configurés.
- Pipeline backend fonctionnel.
- Build frontend fonctionnel.
- Capture d'un pipeline échoué.
- Capture d'un pipeline réussi.
- Court rapport de diagnostic expliquant l'erreur et sa correction.